

Process Log da Solução - Case G4 Churn

Executive Summary

Enfrentamos um paradoxo de churn na RavenStack onde clientes cancelavam a assinatura, mas o uso da plataforma parecia estar crescendo, e o suporte estava com índices positivos. Ao consolidar 5 bases de dados fragmentadas e isolar o comportamento de uso de curto prazo (últimos 30 dias), descobrimos que os clientes em *churn* não estavam usando melhor a ferramenta, mas sim lutando contra erros sistêmicos constantes, mascarados pelo alto engajamento ilusório. Construímos e entregamos um Dashboard interativo em Flask com uma IA nativa (LangChain) para que qualquer liderança consiga dialogar e auditar os ofensores de churn e o faturamento em risco em tempo real.

Processo e Bastidores da IA

Quais ferramentas de IA você usou e por quê? 1. **Antigravity (Agente Codificador):** Utilizado para atuar como par do usuário na engenharia de dados (ETL), treinamento de modelos de Machine Learning locais (Scikit-Learn) e construção *Full Stack* (Frontend e Backend). 2. **LangChain (Pandas DataFrame Agent) + OpenAI (GPT-4o-mini):** Utilizados diretamente no *produto final* (Dashboard). O LangChain permite que a LLM execute código dinâmico e interprete CSVs na mosca, proporcionando ao CEO uma interface de chat capaz de fazer consultas estatísticas complexas sem que ele precise saber SQL.

Como você decompôs o problema antes de promptar? A decomposição seguiu 4 fases táticas: 1. **Unificação (Engenharia):** Mapear chaves primárias e juntar os 5 CSVs para formar uma única *Master Table*. 2. **Decomposição Temporal (O Paradoxo):** Quebrar o engajamento vitalício da plataforma em engajamento *recente* vs *histórico*. Isso provou que o uso alto no momento do churn era, na verdade, tentativas exaustivas devido a bugs e falta de valor percebido. 3. **Modelagem Preditiva:** Ao invés de *achismos*, treinei um modelo *Random Forest* para ditar quais colunas matematicamente tinham maior peso no churn. 4. **Camada Visual:** Focar os gráficos nos ofensores listados pelo modelo (Indústria "DevTools" e uso intenso vs Erros).

Onde a IA errou e como você corrigiu? - Erro de Escopo Geográfico: O modelo tentou, no início, correlacionar o churn com o campo "Country" (país), mas o volume de dados seccionado por país era esparso demais, gerando *overfitting* de análise. **Correção:** Forçamos o foco da análise para campos comportamentais (como `usage_last_30`) através de seleção estrita de *features* no Scikit-Learn. - **Limitações de Ambiente:** Houve falhas pontuais no *build* das bibliotecas de IA via terminal em *background* devido aos caminhos (PATH) bloqueados no Windows. **Correção:** Contornei passando comandos via binário absoluto (`python -m pip install`).

O que você adicionou que a IA sozinha não faria? A Inteligência Artificial pura, treinada para classificar risco de churn, trataria todos os clientes como "1 ou 0". A intervenção de raciocínio de negócios foi cruzar essa probabilidade estatística com o campo `current_mrr` (Monthly Recurring Revenue). Com isso, criamos a visão de **Risco de Caixa**. Em vez de focar na redução puramente volumétrica de clientes, o foco virou "Como salvar o faturamento da RavenStack", ordenando as ações por impacto financeiro. Além disso, a IA generativa tende a entregar designs estáticos; minha intervenção arquitetural foi forçar a migração de um Streamlit simplista para um Flask customizado, com identidade visual da **G4 Business School** e arquitetura cliente-servidor escalável.

Quantas iterações foram necessárias? Foram necessárias cerca de **5 iterações amplas**: 1) Tratamento de dados e unificação; 2) Refutação da hipótese de preço vs erros; 3) Treinamento do modelo preditivo; 4) Primeira versão do Dashboard (Streamlit); 5) Migração para a arquitetura Web Premium (Flask/Vanilla CSS).

Formato da Solução

Abordagem

Process Log - Case G4 Churn

Abordamos o problema usando análise exploratória multivariada em Python e modelagem de risco iterativa. O problema central era o paradoxo das métricas ("CSAT alto, mas alto churn"). Identificamos que o CSAT estava mascarado por clientes silenciosos e que o alto engajamento era, de fato, engajamento com erros da plataforma.

Resultado

1. **Identificação da Causa-Raiz:** O churn é impulsionado por um teto de valor e usabilidade. Clientes da indústria DevTools e FinTech experimentam falhas de integração.
2. **Dashboard de Resolução:** Entrega de um Web App moderno rodando localmente (Backend em Flask, UI em HTML/CSS) com uma LLM embutida (LangChain). Essa interface permite observar os gráficos de risco e consultar a base de dados via chat (conversacional).

Recomendações

- **Curto Prazo:** Desenhar um plano de ação (War Room) focada nas 15 contas com maior MRR em risco que foram mapeadas no cruzamento de dados.
- **Médio Prazo:** O time de Produto precisa revisar os logs de erro da plataforma na indústria "DevTools". Eles estão pagando caro, usando intensivamente e cancelando por incapacidade técnica da ferramenta.
- **Longo Prazo:** Implementar alertas automáticos no CRM que disparem sempre que um cliente de alto MRR aumentar em mais de 30% seus chamados de bugs em menos de 15 dias.

Limitações

- A amostra de cancelamentos (Churn Events) contava com 600 registros, o que limitou um pouco o treinamento de redes neurais profundas, nos restringindo a usar o Random Forest (que atende bem, mas tem teto preditivo).
- Não fomos capazes de analisar os textos puros (análise de sentimentos) dos tickets de suporte, pois o dataset possuía apenas metadados quantitativos (CSAT numérico, first_response_time). Textos originais agregariam muito valor à LLM no chat.